SESSION 7

GROUP BY & HAVING

CRITICAL ANALYTICS TOPIC

Goal: Solve all four tasks using aggregate functions, GROUP BY, and HAVING, with SQL in grey code boxes and simple explanations.

Core Concept: GROUP BY and HAVING

GROUP BY collects rows that share the same value, such as the same user_id, payment_method, genre, or another category. Aggregate functions such as COUNT() and SUM() then calculate one result for each group.

Clause / Function	Purpose
GROUP BY	Creates groups of rows that share the same value.
COUNT(*)	Counts the rows in each group.
SUM(column)	Adds numeric values in each group.
WHERE	Filters individual rows before grouping.
HAVING	Filters groups after aggregate calculations.

Easy rule: Use WHERE to filter rows before grouping. Use HAVING when the condition depends on an aggregate such as COUNT() or SUM().

Session Demo — Total Revenue per Region Above 1M

```
SELECT
    region,
    SUM(revenue) AS total_revenue
FROM sales
GROUP BY region
HAVING SUM(revenue) > 1000000;
```

This example assumes a table named sales with columns region and revenue, and the revenue values use the same currency unit. GROUP BY creates one row per region; HAVING keeps only regions whose total revenue is greater than 1,000,000.

Task 1 — Count Orders per User

Question: Display the total number of orders placed by each user in the food_orders table, grouped by user_id.

```
SELECT
    user_id,
    COUNT(*) AS total_orders
FROM food_orders
GROUP BY user_id;
```

Explanation

- user_id: identifies each user.
- COUNT(*): counts the order rows for each user.
- GROUP BY user_id: creates one group for each user, so the count is calculated separately per user.

Note: Because the query starts from food_orders, it returns users who appear in that table. To include users with zero orders, a separate users table and a LEFT JOIN would be needed.

Task 2 — Total Amount by Payment Method

Question: Using transactions(transaction_id, user_id, amount, payment_method), show the total amount spent through each payment method.

```
SELECT
    payment_method,
    SUM(amount) AS total_amount
FROM transactions
GROUP BY payment_method;
```

Explanation

- payment_method: identifies the payment category, such as card, cash, or UPI.
- SUM(amount): adds all transaction amounts within that payment method.
- GROUP BY payment_method: returns one row per payment method.

Illustrative Output

payment_method	total_amount
UPI	12500.00
Card	8400.00
Cash	2100.00

The values above are only an example. Actual totals depend on the records in the transactions table.

Task 3 — Genres with Box Office Collection Above 10 Crore

Question: From movies(movie_id, genre, box_office_collection), show each genre and its total box office collection, but only when that total is above 10 crore.

```
SELECT
    genre,
    SUM(box_office_collection) AS total_collection
FROM movies
GROUP BY genre
HAVING SUM(box_office_collection) > 100000000;
```

Explanation

- genre: identifies each movie genre.
- SUM(box_office_collection): adds the collections of all movies in a genre.
- GROUP BY genre: makes one group per genre.
- HAVING SUM(box_office_collection) > 100000000: displays only genres with collection greater than 10 crore, assuming box_office_collection is stored in rupees.

Conversion: 1 crore = 10,000,000. Therefore, 10 crore = 100,000,000 rupees. If the column stores values directly in crores instead of rupees, use HAVING SUM(box_office_collection) > 10.

Why use HAVING instead of WHERE?

The condition checks the SUM() of each genre. Since the total is calculated after grouping, HAVING is the correct clause for filtering those totals.

Task 4 — Users with More Than Two Hours of Playlist Duration

Question: Using playlist(playlist_id, user_id, song_id, duration), find users whose combined song duration is more than 2 hours. Duration is measured in seconds.

```
SELECT
    user_id,
    SUM(duration) AS total_duration
FROM playlist
GROUP BY user_id
HAVING SUM(duration) > 7200;
```

Explanation

- user_id: identifies the user who created the playlist entries.
- SUM(duration): adds song durations for each user.
- GROUP BY user_id: calculates a separate total for each user.
- HAVING SUM(duration) > 7200: keeps users whose combined duration exceeds 7,200 seconds.

Time conversion: 60 seconds × 60 minutes × 2 hours = 7,200 seconds. The query uses > 7200, so exactly 2 hours does not qualify; the total must be greater than 2 hours.

Final Revision — Remember These Patterns

Requirement	SQL pattern
Count rows per user	COUNT(*) + GROUP BY user_id
Total amount per category	SUM(amount) + GROUP BY category
Filter groups by total	HAVING SUM(amount) > threshold
Filter groups by count	HAVING COUNT(*) > threshold
Filter individual rows before grouping	WHERE condition, before GROUP BY

One-line Memory Trick

GROUP BY: Decides how to make groups. Aggregate function: calculates a value for each group. HAVING: decides which groups remain in the final result.